

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Organización de Lenguajes y Compiladores 1 "A"
Ing. Mario Jose Bautista Fuentes
Aux.Carlos Fernando Enrique López García



Manual de usuario Fase 1 Proyecto

Estudiante	Carné
Ronald Renán Samayoa Martínez	202031046

Guatemala 12 de junio de 2026

OBJETIVO

Otorgar un soporte al usuario del software del proyecto, para tener un control e informacion oportuna sobre el manejo del programa. Cabe resaltar que el programa está desarrollado para tener únicamente a un usuario, el cual tiene la posibilidad de abrir varias pestañas a la vez por ejecucion del sistema. El programa implementa una interfaz gráfica para una mayor comodidad con el usuario.

Prerrequisitos

- Mínimo 4 GB de memoria RAM
- Sistema operativo, preferentemente Linux o Windows
- Se necesita la instalacion de una version de Java. Una de las consideraciones importantes durante la instalación es el orden en el que se instalan los componentes de Java, específicamente el JDK (Java Development Kit) y el JRE (Java Runtime Environment), ademas de tener una version de maven instalada.

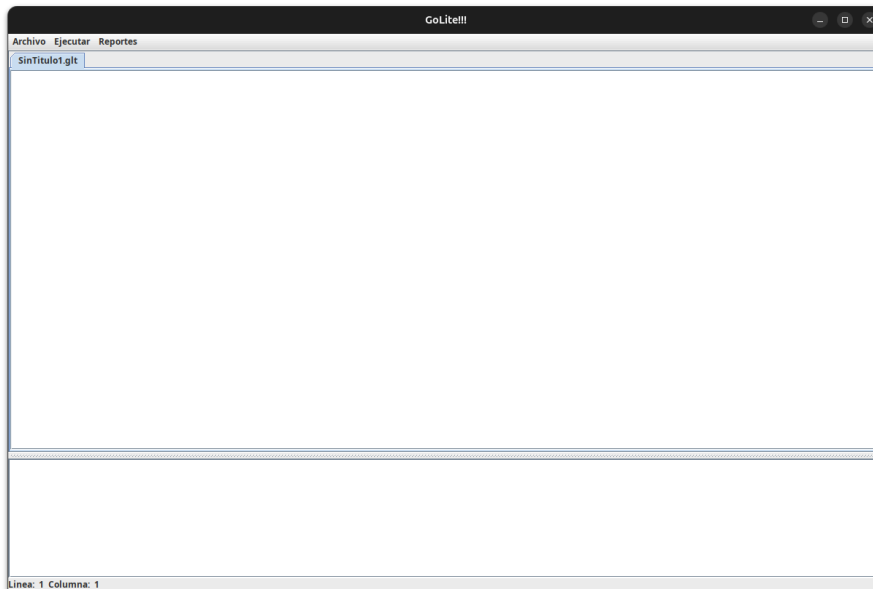
Observacion

Cualquier documento que se desee ingresar en el programa debera tener la extension .glt, esto por recomendaciones tecnicas y mejor manejo de los documentos del lenguaje GoLite

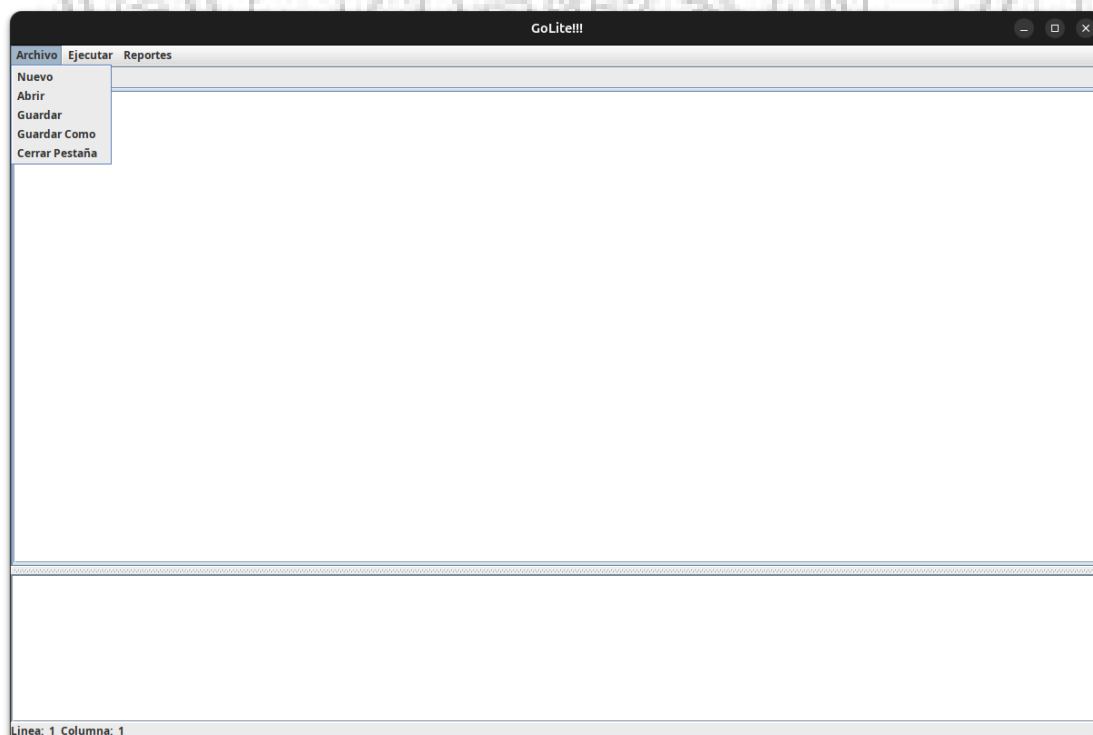


INICIO DEL PROGRAMA

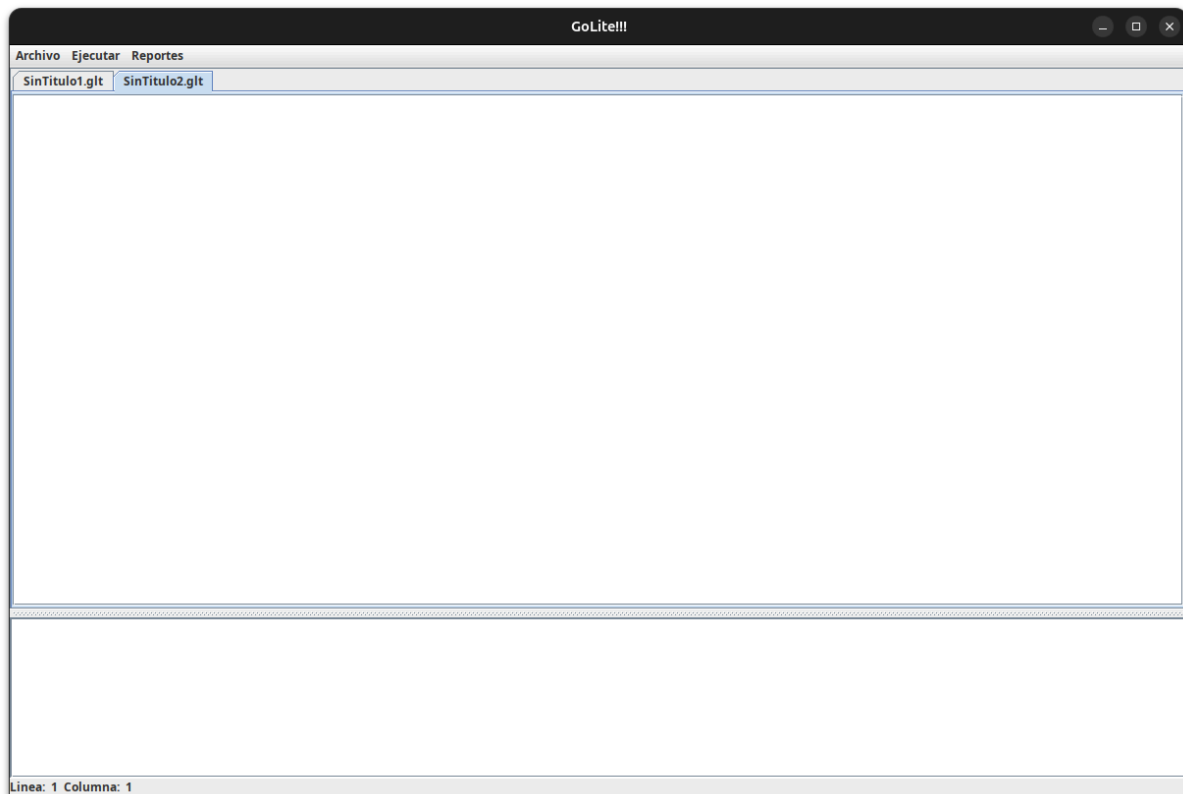
Al ejecutar, vemos la ventana principal, en la cual podemos observar detalles como un panel de edicion para poder ingresar texto, una consola que ayuda a mostrar los resultados de los análisis, y arriba de todo una barra de menús con las opciones: Archivo, Ejecutar y Reportes. Asimismo, al iniciar el programa se ejecuta automaticamente un nuevo archivo, que por defecto lleva el nombre de “SinTitulo1.glt”



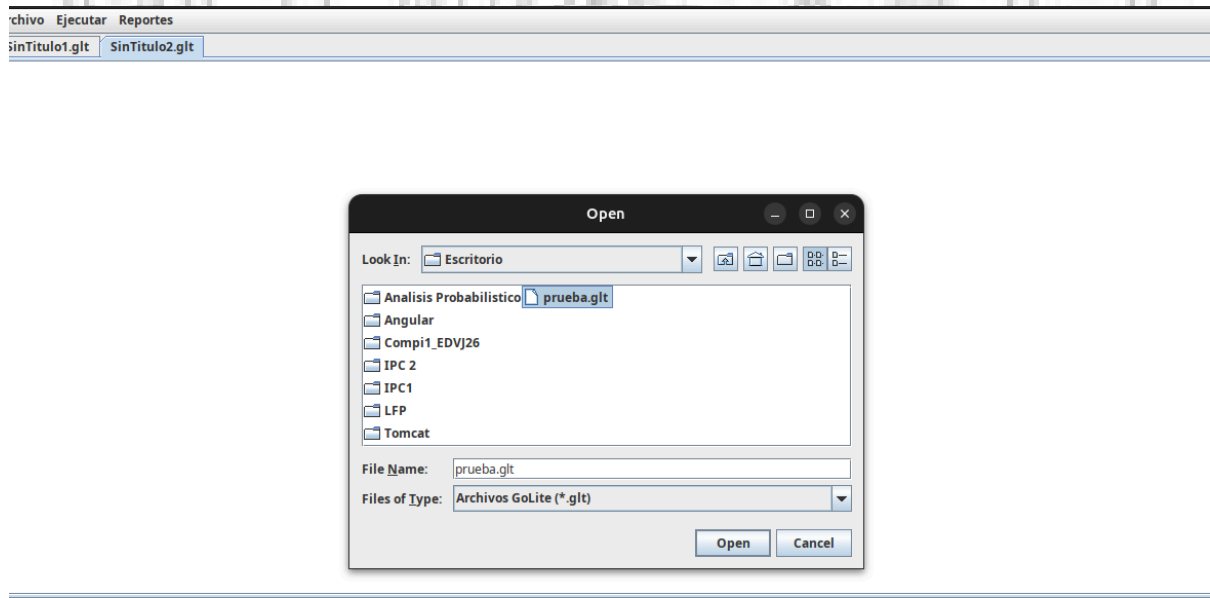
Al seleccionar en la barra la opcion de Archivo se despliegan las opciones: Nuevo, Abrir, Guardar, Guardar Como, y Cerrar Pestaña

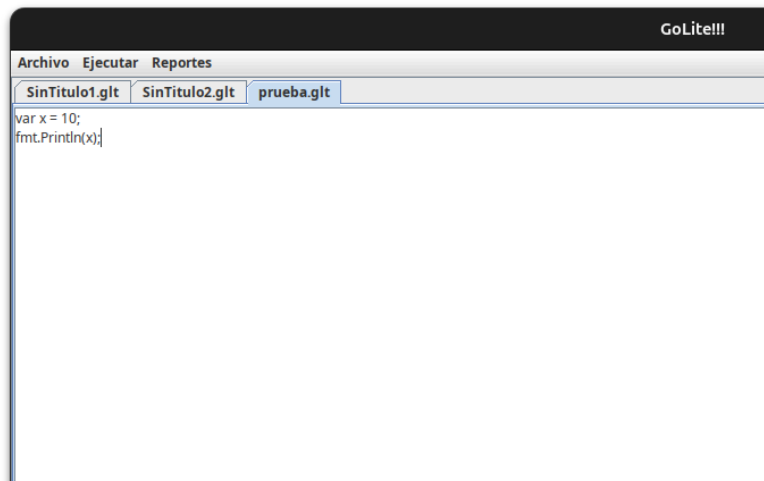


Si selecciona “Nuevo”, se abrirá una nueva pestaña, lleva el nombre de “SinTitulo2.glt”

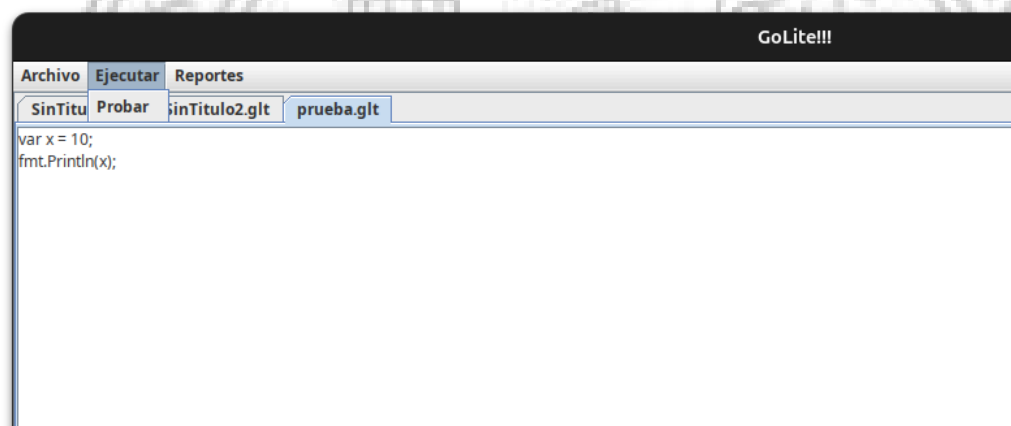


Si selecciona “Abrir”, una nueva pestaña se abre y lleva el nombre del archivo correspondiente (el cual debe estar en formato .glt para poder ser aceptado)



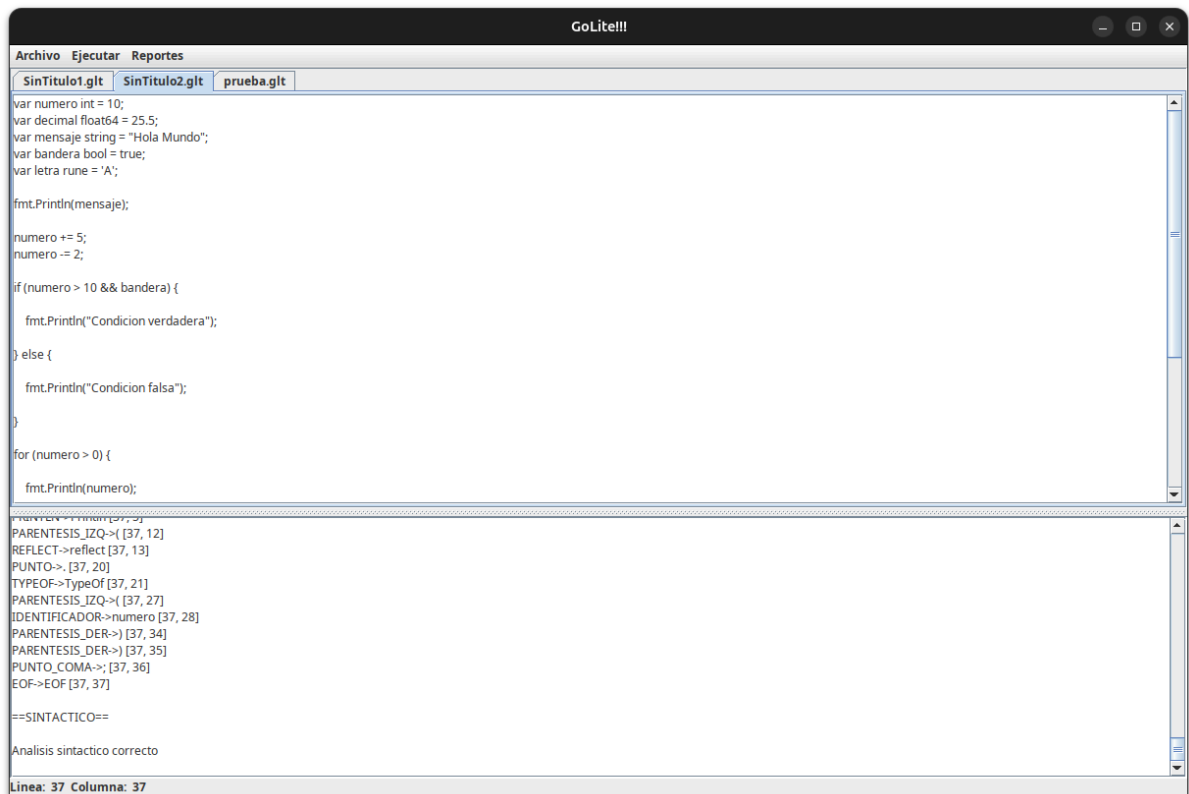


Al seleccionar en la barra la opción de Ejecutar se despliega la opción Probar, la cual procederá a realizar el análisis léxico y sintáctico en el texto introducido en el panel respectivo



Al ejecutar el análisis, se muestra en consola el resultado del texto ingresado, pudiendo ver tokens y un mensaje de análisis sintáctico correcto o con errores

Mensaje de análisis sintáctico correcto:



The screenshot shows the GoLite!!! IDE interface. The top menu bar includes 'Archivo', 'Ejecutar', and 'Reportes'. Below the menu is a tab bar with three tabs: 'SinTitulo1.glt', 'SinTitulo2.glt', and 'prueba.glt'. The main editor area displays a Go program. The bottom panel shows the output of the syntax analysis, which is 'Analisis sintactico correcto'. The status bar at the bottom indicates 'Linea: 37 Columna: 37'.

```
var numero int = 10;
var decimal float64 = 25.5;
var mensaje string = "Hola Mundo";
var bandera bool = true;
var letra rune = 'A';

fmt.Println(mensaje);

numero += 5;
numero -= 2;

if (numero > 10 && bandera) {

    fmt.Println("Condicion verdadera");

} else {

    fmt.Println("Condicion falsa");

}

for (numero > 0) {

    fmt.Println(numero);

}

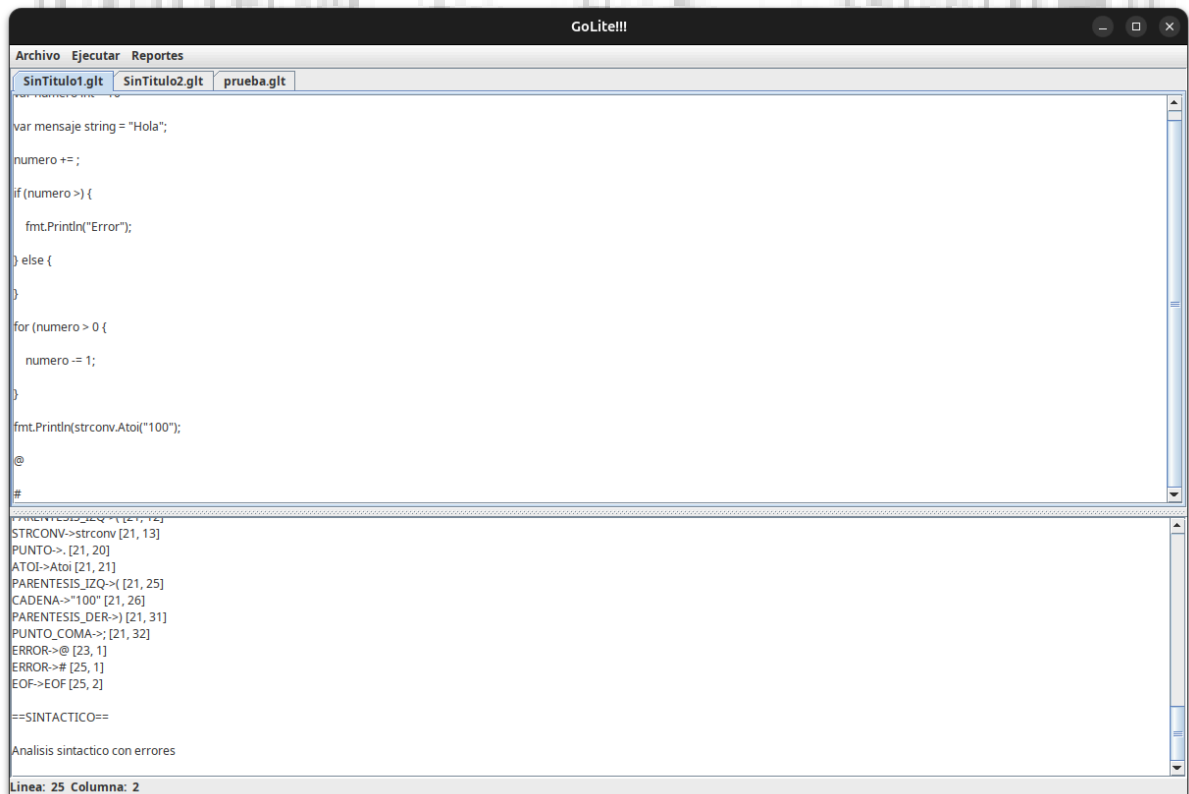
PARENTESIS_IJQ->{ [37, 12]
REFLECT->reflect [37, 13]
PUNTO->.[37, 20]
TYPEOF->typeof [37, 21]
PARENTESIS_IJQ->{ [37, 27]
IDENTIFICADOR->numero [37, 28]
PARENTESIS_DER-> } [37, 34]
PARENTESIS_DER-> } [37, 35]
PUNTO_COMA->.[37, 36]
EOF->EOF [37, 37]

==SINTACTICO==

Analisis sintactico correcto

Linea: 37 Columna: 37
```

Mensaje de análisis sintactico correcto:



The screenshot shows the GoLite!!! IDE interface. The top menu bar includes 'Archivo', 'Ejecutar', and 'Reportes'. Below the menu is a tab bar with three tabs: 'SinTitulo1.glt', 'SinTitulo2.glt', and 'prueba.glt'. The main editor area displays a Go program. The bottom panel shows the output of the syntax analysis, which is 'Analisis sintactico con errores'. The status bar at the bottom indicates 'Linea: 25 Columna: 2'.

```
var mensaje string = "Hola";

numero += ;

if (numero >) {

    fmt.Println("Error");

} else {

}

for (numero > 0 {

    numero -= 1;

}

fmt.Println(strconv.Atoi("100"));

@
#

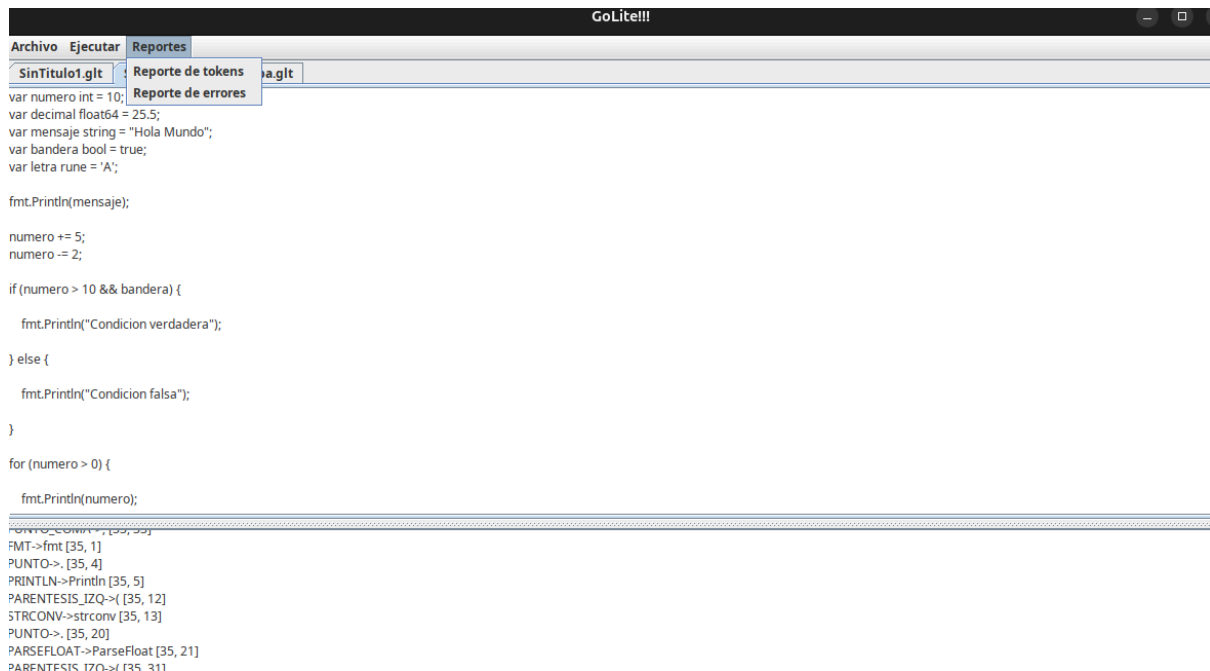
STRCONV->strconv [21, 13]
PUNTO->.[21, 20]
atoi->Atoi [21, 21]
PARENTESIS_IJQ->{ [21, 25]
CADENA->"100" [21, 26]
PARENTESIS_DER-> } [21, 31]
PUNTO_COMA->.[21, 32]
ERROR->@ [23, 1]
ERROR-># [25, 1]
EOF->EOF [25, 2]

==SINTACTICO==

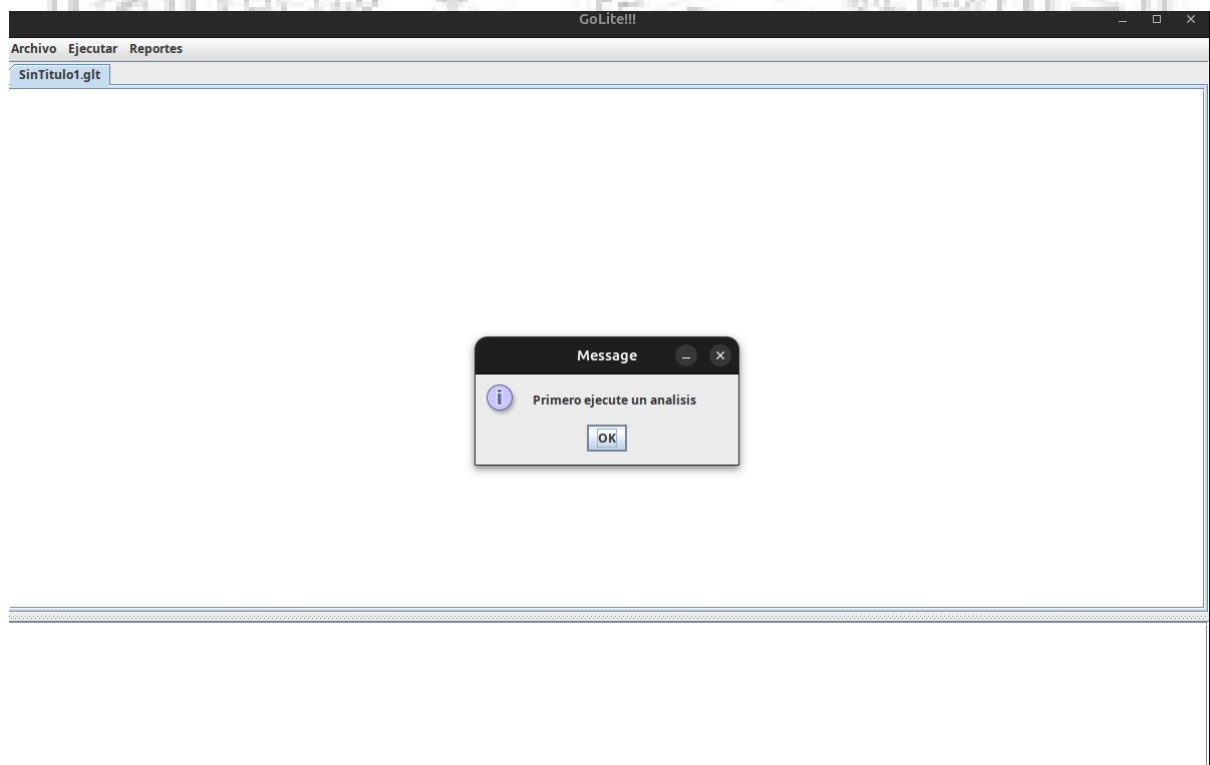
Analisis sintactico con errores

Linea: 25 Columna: 2
```

Al seleccionar en la barra la opción de reportes, podremos elegir entre mostrar el reporte de tabla de tokens obtenidos y el reporte de errores tanto léxicos como sintácticos



Si se intenta hacer un reporte sin haber hecho un análisis, el sistema pide que primero hagamos un análisis para poder generar el reporte como tal



Ejemplo de reporte de tokens

GoLite!!!

Archivo Ejecutar Reportes

SinTitulo1.glt SinTitulo2.glt prueba.glt

```

var numero int = 10

var mensaje string = "Hola";

numero += ;

if (numero >) {

    fmt.Println("Error");

} else {

}

for (numero > 0 {

    numero -= 1;

})

fmt.Println(strconv.Atoi("100");

@

```

==SINTACTICO==

Analisis sintactico con errores

Reporte de Tokens

Tipo	Lexema	Linea	Columna
VAR	var	1	1
IDENTIFICADOR	numero	1	5
INT	int	1	12
IGUAL	=	1	16
ENTERO	10	1	18
VAR	var	3	1
IDENTIFICADOR	mensaje	3	5
STRING	string	3	13
IGUAL	=	3	20
CADENA	"Hola"	3	22
PUNTO_COMA	;	3	28
IDENTIFICADOR	numero	5	1
MAS_IGUAL	+=	5	8
PUNTO_COMA	;	5	11
IF	if	7	1
PARENTESIS_IZQ	{	7	4
IDENTIFICADOR	numero	7	5
MAYOR	>	7	12
PARENTESIS_DER	}	7	13
LLAVE_IZQ	{	7	15
FMT	fmt	9	5
PUNTO	.	9	8
PRINTLN	println	9	9
PARENTESIS_IZQ	{	9	16
CADENA	"Error"	9	17
PARENTESIS_DER	}	9	24
PUNTO_COMA	;	9	25
LLAVE_DER	}	11	1
ELSE	else	11	3
LLAVE_IZQ	{	11	8
LLAVE_DER	}	13	1
FOR	for	15	1
PARENTESIS_IZQ	{	15	5
IDENTIFICADOR	numero	15	6

Ejemplo de reporte de errores

GoLite!!!

Archivo Ejecutar Reportes

SinTitulo1.glt SinTitulo2.glt prueba.glt

```

var numero int = 10

var mensaje string = "Hola";

numero += ;

if (numero >) {

    fmt.Println("Error");

} else {

}

for (numero > 0 {

    numero -= 1;

})

fmt.Println(strconv.Atoi("100");

@

```

==SINTACTICO==

Analisis sintactico con errores

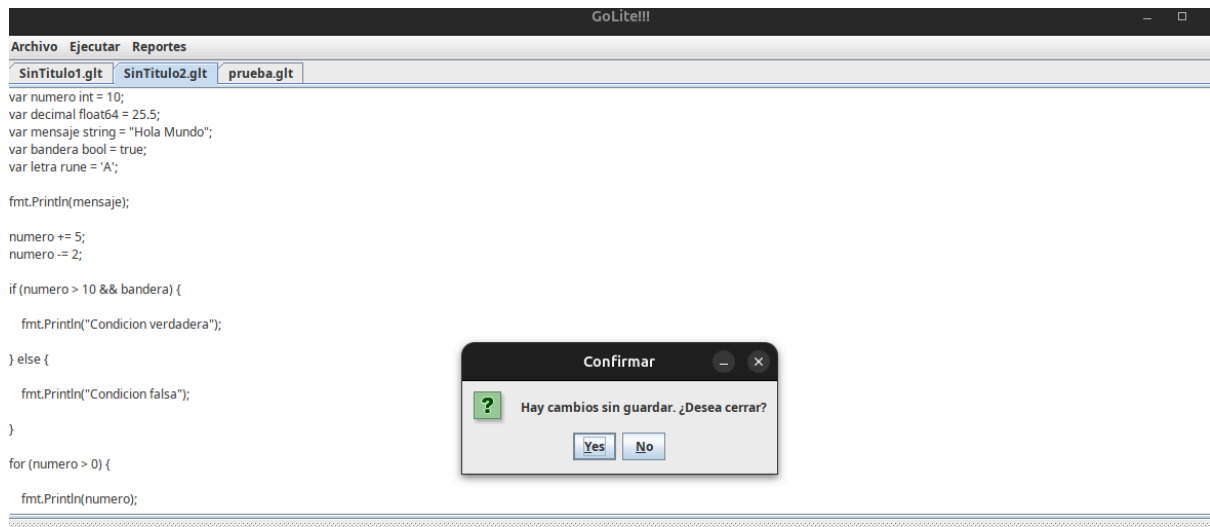
Reporte de Errores

Tipo	Descripcion	Linea	Columna
Lexico	Caracter no reconocido: @	23	1
Lexico	Caracter no reconocido: #	25	1
Sintactico	Token inesperado: null	3	1
Sintactico	Token inesperado: null	5	11
Sintactico	Token inesperado: null	7	13

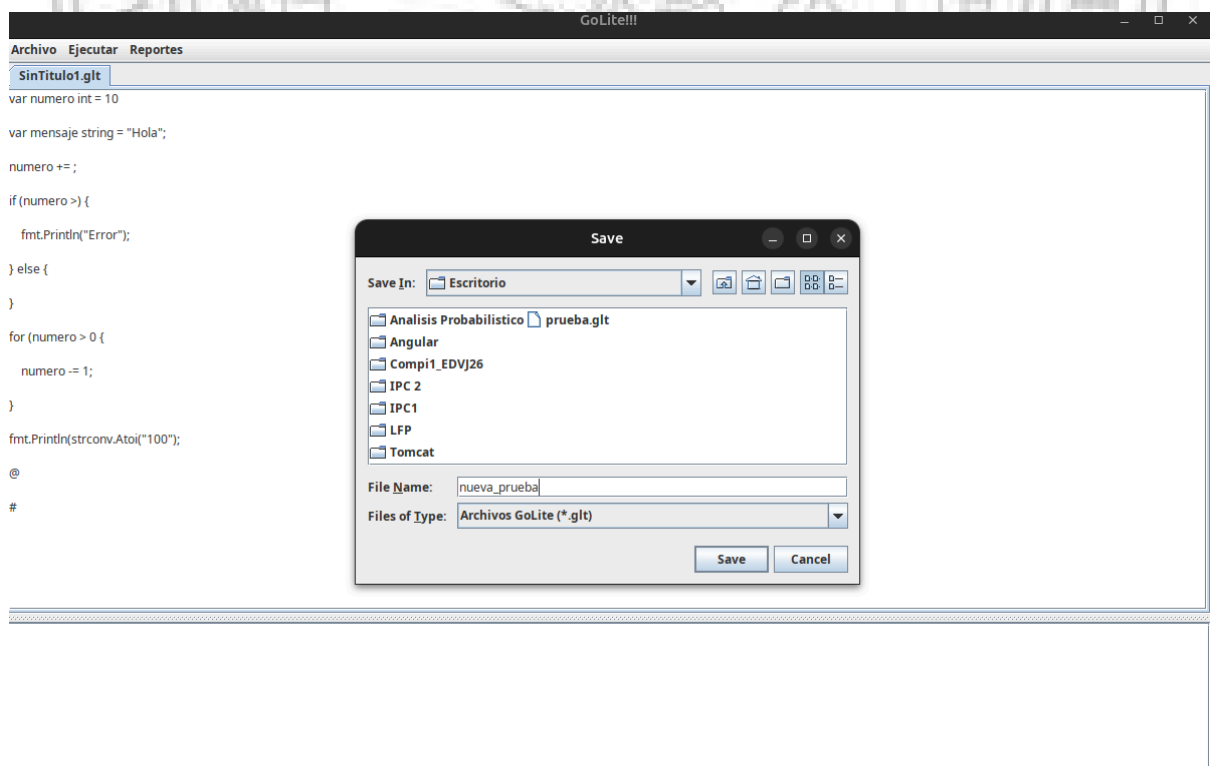
Si se selecciona la opcion de cerrar pestaña o se presiona la opcion de salir del programa pero no se han guardado cambios, el sistema lanza el mensaje de advertencia, pudiendo pensar si

en verdad se quiere salir del programa.

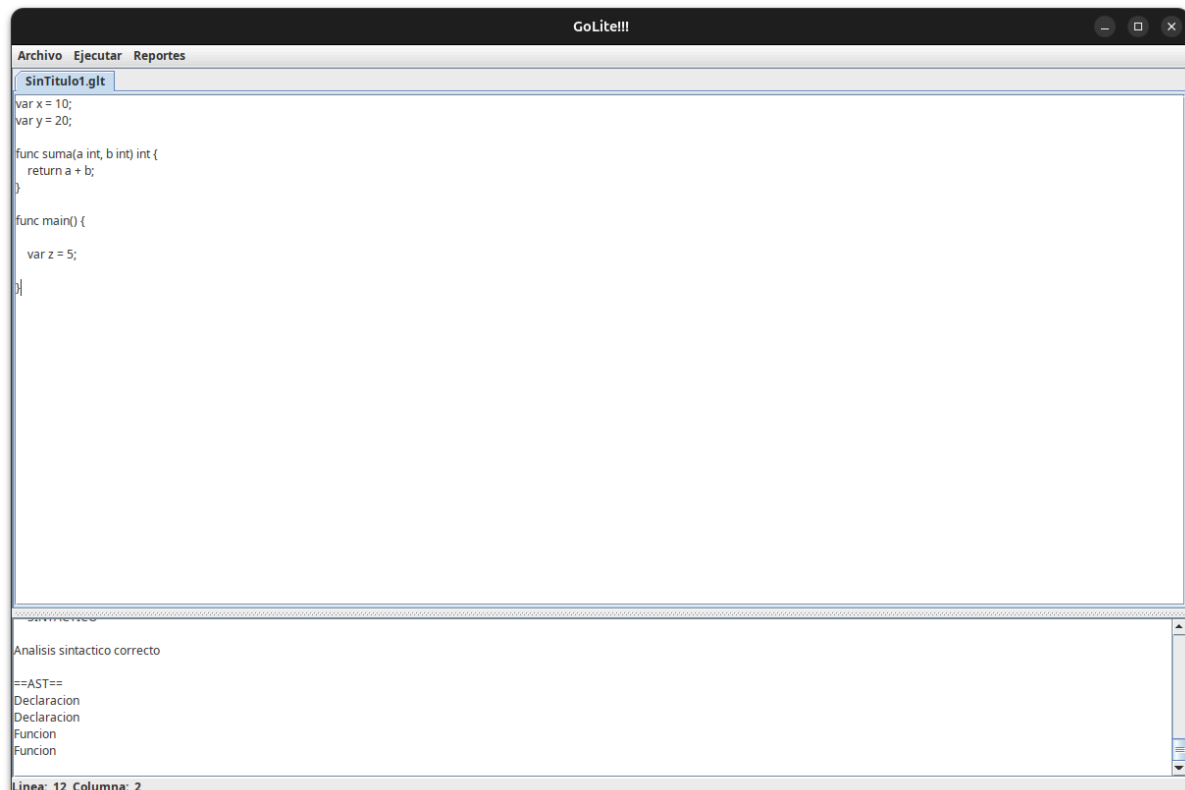
Si se indica que sí, se cierra sin ningún problema; mientras que si se elige que no, el programa mantiene la pestaña abierta



Finalmente, si se desean guardar los cambios, el sistema nos permite elegir dónde guardarlo y colocarle un nombre, este archivo puede en formato .glt por defecto o en el formato All Files



Para la fase 2, fue implementada la funcion de analisis semantico, con la cual podemos asignar valores a las variable, realizar operaciones algebraicas, entre otras funciones



The screenshot shows the GoLite!!! IDE with a menu bar (Archivo, Ejecutar, Reportes) and a toolbar. The main editor displays a Go program named 'SinTitulo1.glt' with the following code:

```
var x = 10;
var y = 20;

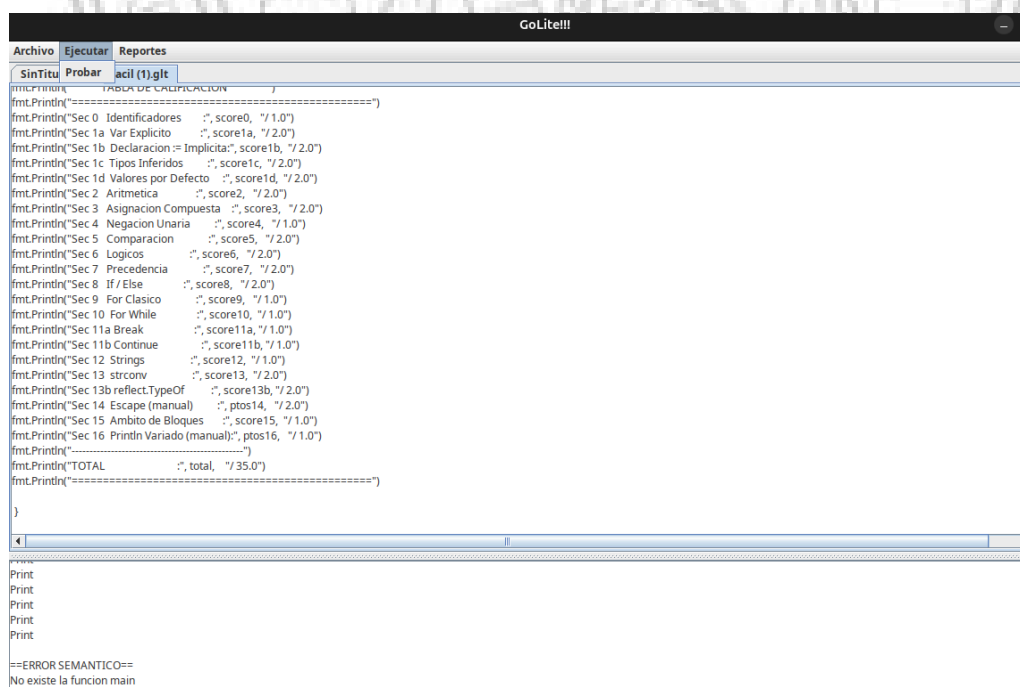
func suma(a int, b int) int {
    return a + b;
}

func main() {
    var z = 5;
}
```

The output pane at the bottom shows the results of the semantic analysis:

```
Analisis sintactico correcto
==AST==
Declaracion
Declaracion
Funcion
Funcion
Linea: 12 Columna: 2
```

Si no existe una funcion main, el analisis semantico lo reporta, no permitiendo funcionar de la manera correcta



The screenshot shows the GoLite!!! IDE with a menu bar (Archivo, Ejecutar, Reportes) and a toolbar. The main editor displays a Go program named 'SinTitulo1.glt' with the following code:

```
func suma(a int, b int) int {
    return a + b;
}

func main() {
    var z = 5;
}
```

The output pane at the bottom shows the results of the semantic analysis, including a table of scores and a final error message:

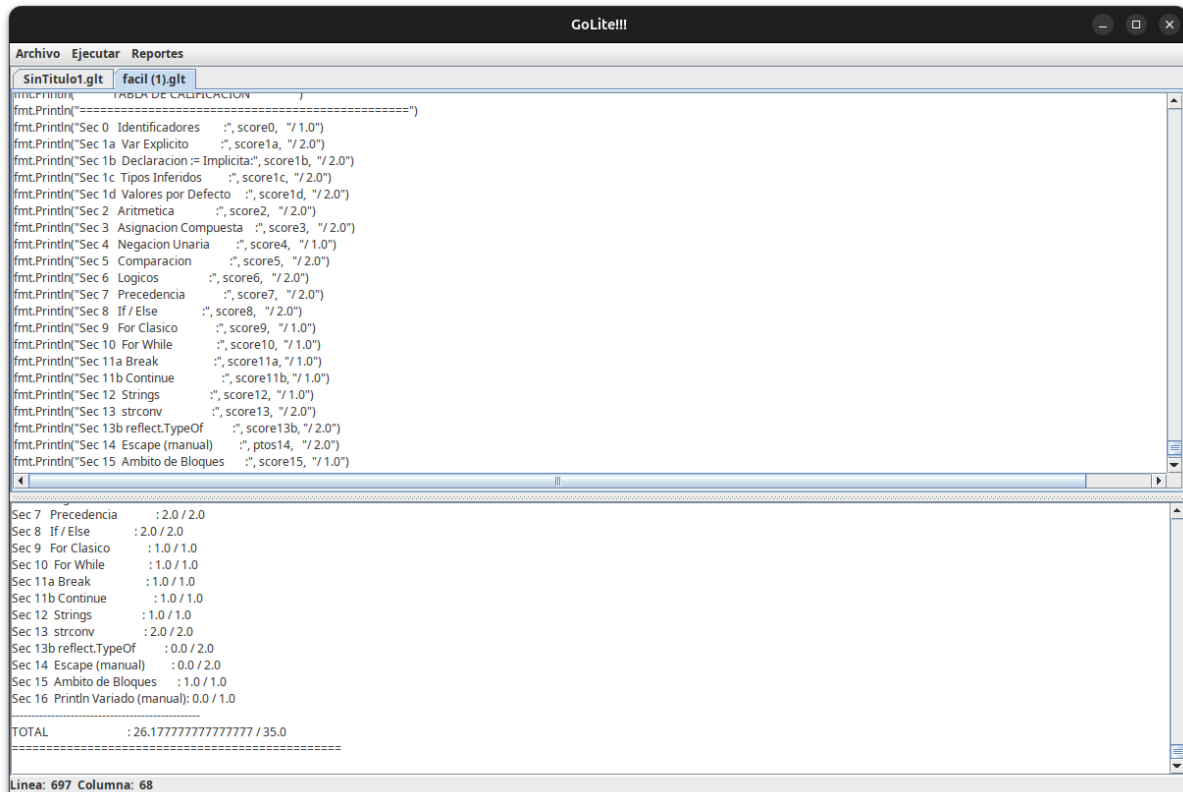
```
func suma(a int, b int) int {
    return a + b;
}

func main() {
    var z = 5;
}
```

Identificadores	score0	score0
Var Explicito	score1a	score1a
Declaracion := Implicita	score1b	score1b
Tipos Inferidos	score1c	score1c
Valores por Defecto	score1d	score1d
Aritmetica	score2	score2
Asignacion Compuesta	score3	score3
Negacion Unaria	score4	score4
Comparacion	score5	score5
Logicos	score6	score6
Precedencia	score7	score7
If / Else	score8	score8
For Clasico	score9	score9
For While	score10	score10
Break	score11a	score11a
Continue	score11b	score11b
Strings	score12	score12
strconv	score13	score13
reflect.TypeOf	score13b	score13b
Escape (manual)	ptos14	ptos14
Ambito de Bloques	score15	score15
Println Variado (manual)	ptos16	ptos16
TOTAL	total	total

```
==ERROR SEMANTICO==
No existe la funcion main
```

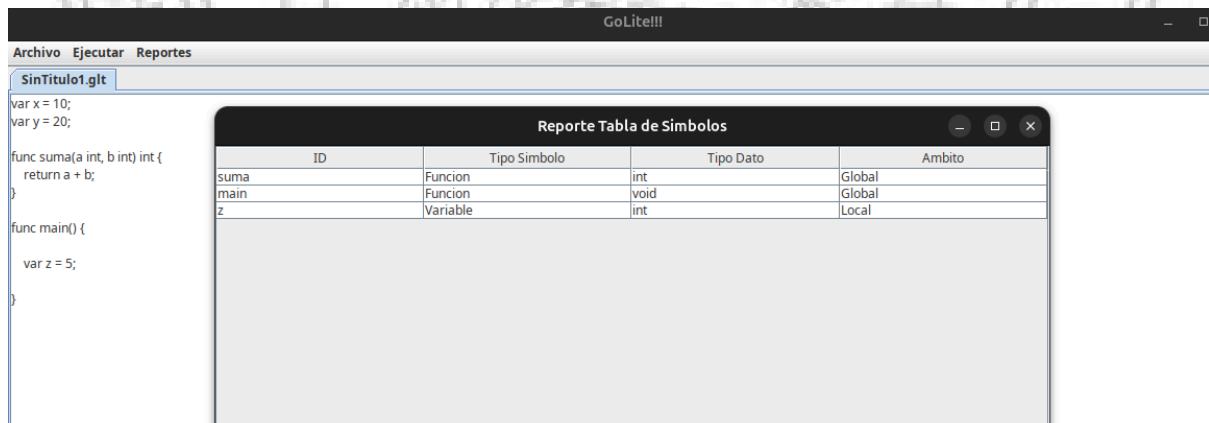
Ejemplo cuando si tiene la funcion main



The screenshot shows the GoLite!!! IDE interface. The 'Reportes' menu is open, and the 'facil (1).glt' report is selected. The report displays a table of scores for various sections, including 'Sec 0 Identificadores', 'Sec 1a Var Explicito', 'Sec 1b Declaracion:= Implicita:', 'Sec 1c Tipos Inferidos', 'Sec 1d Valores por Defecto', 'Sec 2 Aritmetica', 'Sec 3 Asignacion Compuesta', 'Sec 4 Negacion Unaria', 'Sec 5 Comparacion', 'Sec 6 Logicos', 'Sec 7 Precedencia', 'Sec 8 If / Else', 'Sec 9 For Clasico', 'Sec 10 For While', 'Sec 11a Break', 'Sec 11b Continue', 'Sec 12 Strings', 'Sec 13 strconv', 'Sec 13b reflect.TypeOf', 'Sec 14 Escape (manual)', and 'Sec 15 Ambito de Bloques'. The total score is 26.177777777777777 / 35.0.

Section	Score
Sec 0 Identificadores	2.0 / 2.0
Sec 1a Var Explicito	2.0 / 2.0
Sec 1b Declaracion:= Implicita:	2.0 / 2.0
Sec 1c Tipos Inferidos	2.0 / 2.0
Sec 1d Valores por Defecto	2.0 / 2.0
Sec 2 Aritmetica	2.0 / 2.0
Sec 3 Asignacion Compuesta	2.0 / 2.0
Sec 4 Negacion Unaria	2.0 / 2.0
Sec 5 Comparacion	2.0 / 2.0
Sec 6 Logicos	2.0 / 2.0
Sec 7 Precedencia	2.0 / 2.0
Sec 8 If / Else	2.0 / 2.0
Sec 9 For Clasico	1.0 / 1.0
Sec 10 For While	1.0 / 1.0
Sec 11a Break	1.0 / 1.0
Sec 11b Continue	1.0 / 1.0
Sec 12 Strings	1.0 / 1.0
Sec 13 strconv	2.0 / 2.0
Sec 13b reflect.TypeOf	0.0 / 2.0
Sec 14 Escape (manual)	0.0 / 2.0
Sec 15 Ambito de Bloques	1.0 / 1.0
Sec 16 Println Variado (manual)	0.0 / 1.0
TOTAL	26.177777777777777 / 35.0

Al igual se agregó un breve reporte sobre tabla de simbolos que la semantica reconoce, se ejecuta de la misma manera que los reportes de tokens y errores



The screenshot shows the GoLite!!! IDE interface. The 'Reportes' menu is open, and the 'SinTitulo1.glt' report is selected. The report displays a table of symbols, including 'suma', 'main', and 'z'. The table has columns for ID, Tipo Simbolo, Tipo Dato, and Ambito.

ID	Tipo Simbolo	Tipo Dato	Ambito
suma	Funcion	int	Global
main	Funcion	void	Global
z	Variable	int	Local

GoLite!!!

Archivo Ejecutar Reportes

SinTítulo1.glt fácil (1).glt

Reporte Tabla de Símbolos

ID	Tipo Símbolo	Tipo Dato	Ambito
suma	Funcion	int	Global
main	Funcion	void	Global
main	Variable	int	Local
z	Variable	int	Local
main	Funcion	void	Global
ptos0	Variable	int	Local
ptos1a	Variable	int	Local
ptos1b	Variable	int	Local
ptos1c	Variable	int	Local
ptos1d	Variable	int	Local
ptos2	Variable	int	Local
ptos3	Variable	int	Local
ptos4	Variable	int	Local
ptos5	Variable	int	Local
ptos6	Variable	int	Local
ptos7	Variable	int	Local
ptos8	Variable	int	Local
ptos9	Variable	int	Local
ptos10	Variable	int	Local
ptos11a	Variable	int	Local
ptos11b	Variable	int	Local
ptos12	Variable	int	Local
ptos13	Variable	int	Local
ptos13b	Variable	int	Local
ptos15	Variable	int	Local
ptos14	Variable	float64	Local
ptos16	Variable	float64	Local
miVariable	Variable	int	Local
privado	Variable	int	Local
id_con_guión	Variable	int	Local
valor1	Variable	int	Local
id_valido5	Variable	int	Local
MAYUSCULAS	Variable	int	Local
mixCase99	Variable	int	Local
valor	Variable	int	Local

=====

fmt.Println("Sec 0 Identificadores

fmt.Println("Sec 1a Var Explicito

fmt.Println("Sec 1b Declaracion :=

fmt.Println("Sec 1c Tipos Inferidos

fmt.Println("Sec 1d Valores por De

fmt.Println("Sec 2 Aritmetica

fmt.Println("Sec 3 Asignacion Com

fmt.Println("Sec 4 Negacion Unari

fmt.Println("Sec 5 Comparacion

fmt.Println("Sec 6 Logicos

fmt.Println("Sec 7 Precedencia

fmt.Println("Sec 8 If / Else

fmt.Println("Sec 9 For Clasico

fmt.Println("Sec 10 For While

fmt.Println("Sec 11a Break

fmt.Println("Sec 11b Continue

fmt.Println("Sec 12 Strings

fmt.Println("Sec 13 strconv

fmt.Println("Sec 13b reflect.TypeOf

fmt.Println("Sec 14 Escape (manua

fmt.Println("Sec 15 Ambito de Bloc

=====

Sec 7 Precedencia : 2.0 / 2.0

Sec 8 If / Else : 2.0 / 2.0

Sec 9 For Clasico : 1.0 / 1.0

Sec 10 For While : 1.0 / 1.0

Sec 11a Break : 1.0 / 1.0

Sec 11b Continue : 1.0 / 1.0

Sec 12 Strings : 1.0 / 1.0

Sec 13 strconv : 2.0 / 2.0

Sec 13b reflect.TypeOf : 0.0 / 2

Sec 14 Escape (manual) : 0.0 /

Sec 15 Ambito de Bloques : 1.0

Sec 16 Println Variado (manual): 0.0 / 1.0

=====

TOTAL : 26.177777777777777 / 35.0

=====

